

نسخة داخلية – سرية

الوثيقة التقنية الكاملة لبناء منصة سوق

تفاصيل التنفيذ الكامل: الساتك التقني، البنية المعمارية، تصميم قاعدة البيانات، آلية عمل كل خاصية، التفاصيل الأمنية الدقيقة، وخطة التنفيذ الأسبوعية.

الغرض: مرجع تقني داخلي لفريق التنفيذ / المالك

لا يُشارك مع العملاء

0. محتويات الوثيقة

البنية والأدوات	1. الستاك التقني الكامل
Architecture	2. البنية المعمارية العامة
الجدول والعلاقات	3. تصميم قاعدة البيانات
Auth + RBAC	4. نظام المصادقة والصلاحيات
Inventory Logic	5. آلية المنتجات والأكواد الرقمية
Payments	6. تكامل الدفع عبر Binance + التبرعات
Security Deep-Dive	7. التفاصيل الأمنية الكاملة
Repo + CI/CD	8. هيكلية المشروع وخط النشر
4 مراحل	9. الخطة الزمنية التفصيلية (أسبوعياً)
Checklist	10. قائمة تحقق ما قبل الإطلاق

1. الستاك التقني الكامل

كل أداة مذكورة مع سبب اختيارها

الواجهة الأمامية (Frontend)

TanStack Query

Framer Motion

TailwindCSS

TypeScript

Next.js 14 (App Router)

shadcn/ui

Next.js لأنه يدعم Server-Side Rendering (مهم للسيو وسرعة التحميل على الجوال)، Tailwind + Framer Motion لتنفيذ كل الأنيميشن وال hover/floating effects بسهولة وأداء عالي. TanStack Query لإدارة طلبات ال API والتخزين المؤقت من غير تعقيد.

الواجهة الخلفية (Backend)

BullMQ

Redis

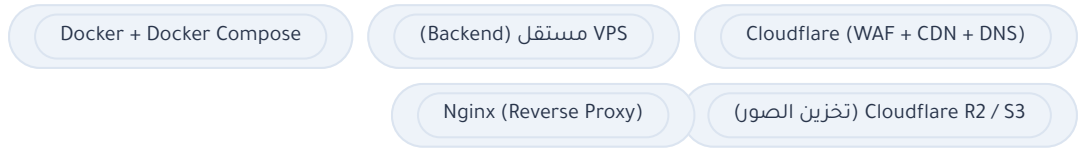
PostgreSQL

Prisma ORM

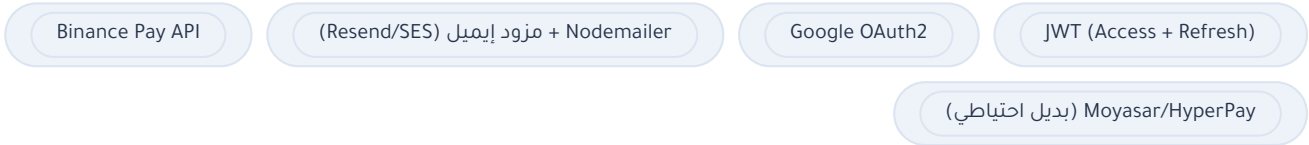
NestJS (Node.js + TypeScript)

NestJS يعطينا بنية منظمة (Modules / Guards / Interceptors) وهذا بالضبط اللي نحتاجه لتطبيق RBAC وطبقات الأمان بشكل نظيف. Prisma يمنع أي SQL Injection لأنه يستخدم Parameterized Queries تلقائياً. Redis لتخزين جلسات الدخول ورموز ال OTP وال Rate Limiting. BullMQ لتشغيل المهام بالخلفية (إرسال الإيميلات، معالجة ال webhooks).

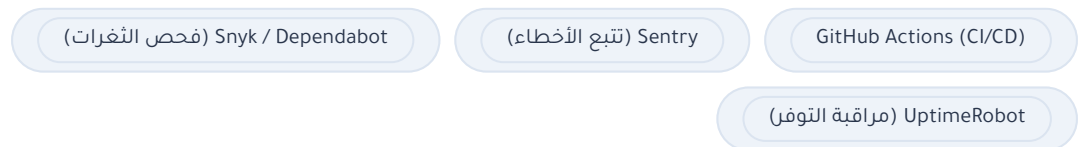
التخزين والبنية التحتية



المصادقة والدفع



المراقبة والجودة



2. البنية المعمارية العامة

نظام مفصول بالكامل (Decoupled): الفرونت يتواصل مع الباك اند فقط عبر REST API محمي بـ JWT. كل طبقة منفصلة فيزيائياً:

1. **Client Layer**: Next.js (يعمل على Vercel أو سيرفر منفصل) – لا يلمس قاعدة البيانات مباشرة أبداً
2. **Edge Layer**: Cloudflare – يستقبل كل الطلبات أولاً (WAF + Rate Limiting + Cache) للملفات الثابتة
3. **Application Layer**: NestJS API على VPS منفصل خلف Nginx. داخل شبكة Docker خاصة غير مكشوفة على الإنترنت مباشرة
4. **Data Layer**: PostgreSQL + Redis – بدون أي وصول خارجي، فقط من داخل شبكة Docker الداخلية
5. **Storage Layer**: S3/R2 منفصل للصور – الباك اند يولّد روابط رفع مؤقتة (Signed URLs) فقط

مبدأ أساسي: قاعدة البيانات لا تُفتح على الإنترنت العام إطلاقاً – الوصول فقط من السيرفر الداخلي عبر شبكة Docker خاصة.

3. تصميم قاعدة البيانات

أهم الجداول والحقول الرئيسية (وليست شاملة كل حقل تقني)

users – المستخدمون (تجار/أفراد)

الحقل	الوصف
id, full_name, email, phone	بيانات أساسية
password_hash	مشقّر بـ Argon2id – فارغ لو الحساب عبر Google
google_id	موجود فقط لحسابات دخول جوجل
is_verified, is_blocked	حالة التحقق والحظر
badge_id	ربط بجدول الشارات (FK)

badges – الشارات/الاعتمادات

الحقل	الوصف
id, name, description, icon, color	يُدار بالكامل من لوحة التحكم

admin_users, roles, permissions, role_permissions, admin_user_roles

الجدول	الوظيفة
admin_users	حسابات الموظفين (منفصلة تماماً عن حسابات العملاء)
roles	مثال: Super Admin, Product Reviewer, Support Agent, Content Editor
permissions	صلاحيات دقيقة مثل: products.approve, users.block, employees.manage
role_permissions / admin_user_roles	جداول ربط تحدد أي موظف يملك أي صلاحية

otp_codes

الحقل	الوصف
code_hash, purpose, expires_at, attempts	الكود يُخزّن مشقّر (Hashed) لا نصي. وينتهي خلال 5 دقائق

categories, products, product_codes

الحقل	الوصف
products: name, description, images[], price, price_after_discount	price_after_discount اختياري
products: status	pending / approved / rejected / blocked
product_codes: code_value_encrypted	مشقّر AES-256-GCM – يُفك التشفير فقط لحظة التسليم للمشتري
product_codes: status	available / reserved / sold

orders, transactions, donations

الجدول	الحقول المهمة
orders	buyer_id, product_id, total_amount, status, payment_provider
transactions	provider_transaction_id, amount, status, raw_payload (JSON للتدقيق)
donations	amount, message, is_public, donor_name (قابل للإخفاء)

cms_content, blog_posts, faqs, audit_logs

الجدول	الوظيفة
cms_content	محتوى صفحة "من نحن" وغيرها – key/value يُعدّل من اللوحة بدون كود
blog_posts, faqs	تُدار بالكامل من لوحة التحكم
audit_logs	سجل غير قابل للتعديل (Insert Only) لكل عملية إدارية: من نفذها، ماذا فعل، من أي IP، متى

4. نظام المصادقة والصلاحيات (كيف يعمل بالتفصيل)

تسجيل مستخدم جديد

المستخدم يدخل بياناته → يُنشأ حساب بحالة "غير مفعل"
يُولد كود OTP من 6 أرقام، يُشفّر ويُخزّن، وتنتهي صلاحيته خلال 5 دقائق (Redis)
يُرسل الكود عبر البريد – محاولات محدودة (3 محاولات / 15 دقيقة) لمنع التخمين
بعد التحقق: الحساب يصبح "مفعل" ويُصدّر له Access Token (صلاحية 15 دقيقة) و Refresh Token (7 أيام، داخل Cookie من نوع httpOnly + secure)

الدخول عبر Google

يتم عبر OAuth2 القياسي – إذا كان البريد موجود مسبقاً يُربط الحساب تلقائياً، وإذا لم يوجد يُنشأ حساب جديد مُفعل مباشرة (لأن جوجل تحقق من البريد بالفعل).

دخول لوحة التحكم (الموظفين/الأدمن)

مهم: كل دخول للوحة التحكم – بلا استثناء – يتطلب OTP، حتى لو الجهاز نفسه دخل قبل دقائق. جلسة اللوحة تنتهي تلقائياً بعد ساعتين من عدم النشاط، والعمليات الحساسة (حظر مستخدم، تعديل صلاحيات موظف) تطلب إعادة تأكيد الهوية (Step-up Authentication).

الدور	مثال صلاحيات
Super Admin	كل الصلاحيات
Product Reviewer	مراجعة المنتجات وقبولها/رفضها فقط
Support Agent	عرض المستخدمين، حظرهم، الرد على الاستفسارات
Content Editor	تعديل "من نحن"، المدونة، الأسئلة الشائعة

كل Endpoint في الباك اند محمي بـ Decorator مخصص مثل `@RequirePermission('products.approve')` – بحيث حتى لو الموظف عرف رابط ال API مباشرة، ما يقدر ينفذ العملية بدون الصلاحية المطلوبة.

5. آلية المنتجات والأكواد الرقمية

دورة حياة المنتج

البائع يضيف المنتج (اسم، تصنيف، صور، وصف، سعر، سعر بعد الخصم اختياري) → الحالة "قيد المراجعة" يظهر في قائمة انتظار لوحة التحكم
المراجع يقبل أو يرفض (مع سبب في حال الرفض)
عند القبول فقط يظهر المنتج للعامة

نظام الأكواد (مثال: كروت شحن)

إذا كان المنتج من نوع "أكواد"، يرفع البائع دفعة أكواد دفعة واحدة، وكل كود يُخزّن كسطر منفصل مشقّر في جدول `product_codes`.

نقطة تقنية حرجية – منع بيع نفس الكود مرتين: عند الشراء، يشغّل النظام معاملة قاعدة بيانات (Database Transaction) مع قفل على مستوى السطر (SELECT ... FOR UPDATE) تحجز كود واحد متاح فقط وتغيّر حالته فوراً إلى "محمّوز"، بحيث لو اشترى شخصان بنفس اللحظة يستحيل يوصلهم نفس الكود.

الكود يظهر للمشتري فقط بعد تأكيد الدفع فعلياً، ويُرسَل أيضاً بالإيميل.

6. تكامل الدفع عبر Binance + التبرعات

مسار الدفع (Binance Pay API)

المشتري يضغط "شراء" → الباك اند ينشئ طلب (Order) بحالة "قيد الانتظار"

استدعاء API "Create Order" Binance Pay بالمبلغ ورقم الطلب الداخلي

يظهر للمشتري رابط/QR للدفع عبر تطبيق Binance

Binance يرسل Webhook للباك اند عند اكتمال الدفع – يتم التحقق من التوقيع (HMAC Signature) قبل قبول أي بيانات منه

بعد التأكد من تطابق المبلغ ورقم الطلب: تُحدَّث حالة الطلب إلى "مدفوع"، ويُشغَّل حِز الكود وإرسال التأكيد تلقائياً
أي طلب غير مدفوع بعد 15 دقيقة يُلغى تلقائياً ويُعاد أي كود محجوز إلى المخزون المتاح

بديل احتياطي: نبني طبقة دفع مجردة (Payment Provider Interface) بحيث يسهل ربط بوابة محلية إضافية (Moyasar/HyperPay) بدون تغيير باقي النظام – مهم لأن التعامل بالعملات الرقمية له اعتبارات تنظيمية متفاوتة بالخليج، فلا تعتمد المنصة على قناة دفع وحيدة.

التبرعات (دعم المنصة مادياً)

نفس منطق الدفع لكن أبسط: المتبرع يدخل المبلغ + الاسم (اختياري "إخفاء الاسم") + رسالة، وبعد نجاح الدفع، إذا اختار الظهور العلني يُضاف تلقائياً لقائمة الداعمين المعروضة في صفحة "من نحن".

7. التفاصيل الأمنية الكاملة (التطبيق الفعلي)

طبقة الشبكة

- Cloudflare Proxy مفعل دائماً – ال IP الحقيقي للسيرفر غير معروف للعمامة
- WAF Managed Rules + قواعد مخصصة لحظر أنماط هجوم معروفة (SQLi/XSS patterns) على مستوى ال Edge قبل ما توصل للسيرفر
- Rate Limiting على مستوى Cloudflare بالإضافة لطبقة تطبيقية ثانية
- Cloudflare Turnstile على نماذج التسجيل، الدخول، وطلب OTP لمنع البوتات

طبقة النقل والتطبيق

- TLS 1.2/1.3 فقط، HSTS مفعل مع Preload
- Helmet.js لضبط رؤوس الحماية (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy)
- NestJS ThrottlerModule: حد أقصى للمحاولات على مسارات حساسة مثل `auth/login/` و `auth/otp/`
- كل مدخلات المستخدم تُتحقق عبر DTOs + class-validator صارمة (`whitelist:true`) بحيث أي حقل غير متوقع يُرفض تلقائياً
- Prisma ORM يمنع SQL Injection بنيته (Parameterized Queries دائماً)
- حماية CSRF عبر SameSite=Strict Cookies + رمز CSRF إضافي للعمليات الحساسة

كلمات المرور والجلسات

- Argon2id لتشفير كلمات المرور (أقوى من bcrypt حالياً)
- Access Token قصير العمر (15 دقيقة) + Refresh Token دوّار مع كشف إعادة الاستخدام (Reuse Detection) – أي محاولة استخدام Refresh Token قديم تُلغى كل جلسات المستخدم فوراً كإجراء احترازي

حماية الملفات المرفوعة

- التحقق من نوع الملف الحقيقي (Magic Bytes) وليس فقط الامتداد
- حد أقصى لحجم الملف، وتخزين بأسماء عشوائية على S3/R2 منفصل تماماً عن السيرفر (بحيث ملف مرفوع لا يقدر ينفذ كسكريبت أبداً)

التشفير والبيانات الحساسة

- أكواد المنتجات (كروت الشحن مثلاً) مشفرة AES-256-GCM على مستوى التطبيق، والمفتاح مخزن خارج الكود (Environment/Secrets Manager) وليس بقاعدة البيانات
- نسخ احتياطي يومي مشفر لقاعدة البيانات، يُخزن في موقع منفصل جغرافياً، واختبار استعادة فعلي شهرياً

المراقبة والتدقيق

- audit_logs يسجل كل عملية إدارية حساسة (من نفّذها، ماذا، من أي IP)
- تنبيه تلقائي عند تكرار محاولات دخول فاشلة أو محاولات OTP خاطئة متكررة (اشتباه هجوم Brute Force) – قفل مؤقت + إشعار للأدمن
- Sentry لتتبع الأخطاء البرمجية فور حدوثها
- Nyk/Dependabot يفحصان المكتبات المستخدمة تلقائياً في كل Push، ويمنعان الدمج لو فيه ثغرة حرجية

اختبار مستمر

- مراجعة أمنية داخلية في نهاية كل مرحلة
- اختبار اختراق خارجي (Penetration Test) قبل الإطلاق الرسمي مباشرة
- تكرار الاختبار كل 3 أشهر بعد الإطلاق

8. هيكلية المشروع وخط النشر (CI/CD)

هيكلية المستودع (مبسّطة)

```
/souq
├── apps/web (Next.js Frontend)
├── apps/api (NestJS Backend)
│   ├── modules/auth
│   ├── modules/products
│   ├── modules/orders
│   ├── modules/payments
│   ├── modules/admin
│   └── modules/audit
├── packages/database (Prisma schema)
└── infra (Docker, Nginx, GitHub Actions)
```

خط النشر (CI/CD) عبر GitHub Actions

- Push إلى develop → تشغيل تلقائي: Type Check + Lint + اختبارات وحدة + فحص أمني للمكتبات
- الدمج إلى staging → نشر تلقائي على سيرفر تجريبي لاختبار الفريق
- الدمج إلى main (بعد موافقة يدوية) → بناء صور Docker → دفعها لسجل الصور → نشر على الإنتاج

9. الخطة الزمنية التفصيلية (تقسيم أسبوعي)

المرحلة 1 – الأساس والتخطيط	أسبوعين
الأسبوع 1: تجهيز المستودع والبنية التحتية (Docker/Cloudflare/VPS)، تصميم قاعدة البيانات الكاملة بـ Prisma، بناء الهوية البصرية وال Wireframes	
الأسبوع 2: بناء وحدة المصادقة كاملة (تسجيل، دخول، OTP، Google)، مع كل طبقات الحماية الأساسية، وربط Cloudflare WAF	
المرحلة 2 – الصفحات والواجهة	3 أسابيع
الأسبوع 1: الرئيسية، الخدمات، المنتجات (عرض وتفاصيل)، شرح الشارات	
الأسبوع 2: المدونة، الأسئلة الشائعة، من نحن (مربوطة بـ CMS)، الشروط والأحكام، أضف منتجك	
الأسبوع 3: كل الأنيميشن/الحركات الانتقالية/ال hover والعناصر العائمة، وضبط التجاوب الكامل مع الجوال	
المرحلة 3 – لوحة التحكم والمدفوعات	3 أسابيع
الأسبوع 1: لوحة مراجعة المنتجات، إدارة الشارات، إدارة المستخدمين (عرض/تعديل/حظر)	
الأسبوع 2: نظام الموظفين والصلاحيات RBAC + OTP إجباري للوحة، نظام Audit Log	
الأسبوع 3: تكامل Binance Pay API (بيئة اختبار Sandbox أولاً)، نظام الأكواد الرقمية، التبرعات	
المرحلة 4 – الاختبار والإطلاق	أسبوعين
الأسبوع 1: اختبار اختراق شامل، اختبار تحمل (Load Test) لمحاكاة 1000-5000 مستخدم، إصلاح أي ثغرات	
الأسبوع 2: مراجعة نهائية على الأجهزة والمتصفحات، إطلاق تجريبي محدود، ثم الإطلاق الرسمي مع تفعيل المراقبة المستمرة (Sentry + UptimeRobot)	

10. قائمة تحقق ما قبل الإطلاق

- كل ال Environment Variables والمفاتيح السرية خارج الكود ولا يوجد أي سر مكتوب داخل المستودع
- HTTPS/HSTS مفعل على كل النطاقات الفرعية
- نسخة احتياطية حديثة تم اختبار استعادتها فعلياً
- كل مسارات لوحة التحكم تتطلب OTP وصلاحيات محددة، وتم اختبارها بحساب موظف محدود الصلاحيات

- اختبار اختراق خارجي منتهي وتم إغلاق كل الثغرات الحرجة والمتوسطة
- Webhook الخاص بـ Binance يتحقق من التوقيع (Signature) ولا يقبل أي طلب غير موقع
- Rate Limiting مفعل على كل مسارات المصادقة والدفع
- سياسة الخصوصية والشروط والأحكام مراجعة قانونياً، وتوضيح آلية التعامل مع المدفوعات الرقمية
- أنظمة المراقبة (Sentry / UptimeRobot) مفعلة وتُرسل تنبيهات فعلية